

PCCST501 – Computer Networks

Assignment 1

How the Internet Delivers a Web Page

Website Chosen: YouTube (<https://www.youtube.com>)

Student Name: Muhammed Aamir

Branch: B.Tech Computer Science and Engineering

Semester: S5

Section: CSE A

Scheme: KTU 2024 Scheme

Module: Module 1

Submitted To: Arsha J K

How the Internet Delivers a Web Page

This report explains, step by step, how a web page is delivered to a user when they access YouTube (<https://www.youtube.com>) through a browser. Each section connects a core networking concept from the TCP/IP suite to what actually happens behind the scenes on YouTube.

1. TCP/IP Protocol Stack and the Role of Each Layer

Every time a browser talks to a web server, the communication does not happen in one single step. It is broken down and handled by different layers of the TCP/IP model, where each layer takes care of one specific job and passes the result to the layer above or below it. This layered design keeps the whole system organised and makes it possible for different technologies to work together without depending on each other's internal details. Figure 1 shows the four layers of the TCP/IP stack used in this report, along with the direction in which data travels while sending and receiving.

TCP/IP Protocol Stack

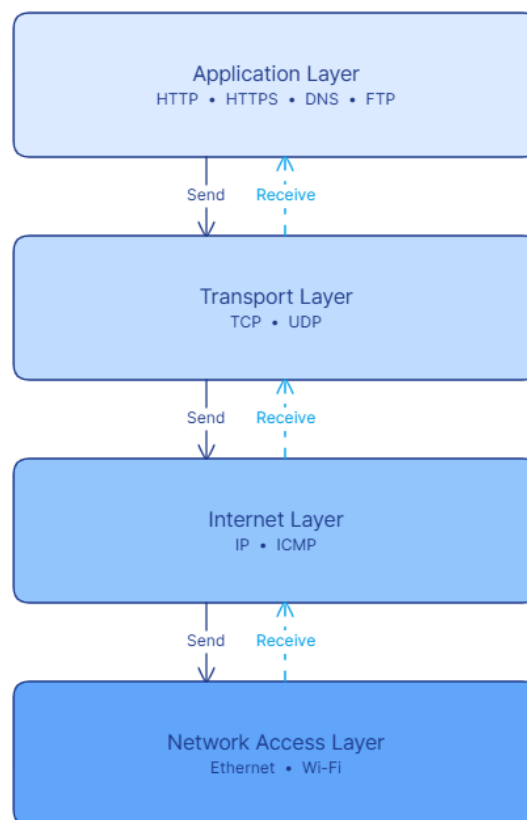


Figure 1: TCP/IP Protocol Stack

Application Layer: This is the topmost layer and the one users indirectly interact with. It contains protocols such as HTTP, HTTPS, DNS and FTP that define how applications like browsers request

and exchange data. When we open YouTube, it is this layer that generates the actual request for the video page.

Transport Layer: This layer is responsible for taking the data from the application layer and delivering it correctly between the two end systems. It uses TCP when reliability is required, as in loading a webpage, and UDP when speed matters more than guaranteed delivery, such as in some parts of video streaming.

Internet Layer: Also called the network layer, this layer handles addressing and routing. It uses IP to give every device a unique address and decides the path packets should take to reach YouTube's servers, while ICMP is used for error reporting and diagnostics like ping.

Network Access Layer: This is the lowest layer and deals with the physical transmission of data over a medium such as Wi-Fi or Ethernet. It converts the packets into frames and signals that can actually travel across the network hardware connecting the user's device to the internet.

2. Role of DNS in Converting the Domain Name to an IP Address

Computers do not understand names like `www.youtube.com` directly; they need numeric IP addresses to locate and connect to a server. The Domain Name System, or DNS, exists exactly for this purpose. It works like a large distributed directory that keeps mapping human-friendly names to their corresponding IP addresses, so that users never have to memorise long strings of numbers.

When a user types `www.youtube.com` in the address bar, the browser first checks whether it already knows the IP address from a previous visit by looking at its own cache. If it is not found there, the operating system's cache is checked next. When the address is still not available locally, the request is passed on to the ISP's DNS resolver, which queries the root DNS server, then the `.com` top-level domain server, and finally the authoritative DNS server for YouTube. The authoritative server returns the correct IP address, which travels back through the same chain until the browser receives it. Figure 2 illustrates this complete lookup process.

DNS Name Resolution Process

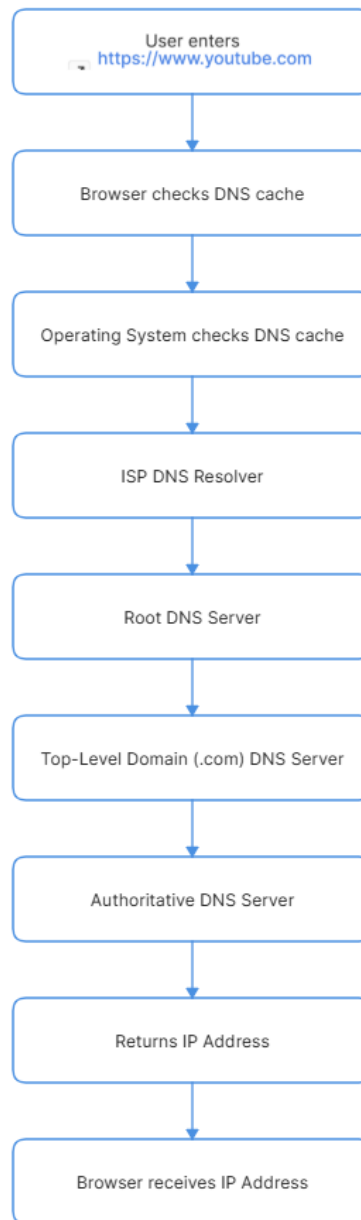


Figure 2: DNS Name Resolution Process for youtube.com

Once the browser has this IP address, it can finally open a connection with YouTube's actual web server and begin the process of requesting the page.

3. How HTTPS Is Used to Request and Receive the Webpage

YouTube uses HTTPS rather than plain HTTP, which means the same request-response exchange takes place but over an encrypted connection using TLS. This protects the data exchanged between the browser and YouTube's servers from being read or altered by anyone intercepting the traffic.

After the IP address is resolved and a TCP connection is set up, the browser sends an HTTPS GET request asking the server for the YouTube homepage. This request includes a request line specifying the method and the resource being asked for, header fields carrying details such as the browser type and accepted content formats, and sometimes cookies identifying the user.

YouTube's web server receives this request, processes it, and prepares an HTTPS response. The response starts with a status line containing a status code, for example 200 to indicate success or 404 if the resource is not found, followed by header fields and the actual body of the response. The body typically contains the HTML structure of the page along with references to CSS files for styling, JavaScript files for interactivity, and links to images and video content that the browser will fetch and display.

Once the browser receives all this data, it parses the HTML, applies the CSS, executes the JavaScript, and loads the images and videos, finally rendering the complete YouTube homepage that the user sees.

4. How TCP Provides Reliable Communication for HTTPS

Before any HTTP or HTTPS request can be sent, TCP first establishes a connection between the browser and the web server through a process known as the three-way handshake. The client sends a SYN segment to the server, the server replies with a SYN-ACK segment, and the client finally sends an ACK segment back. Only after this exchange is the connection considered established, and only then does actual data transfer begin. Figure 3 shows this exchange.

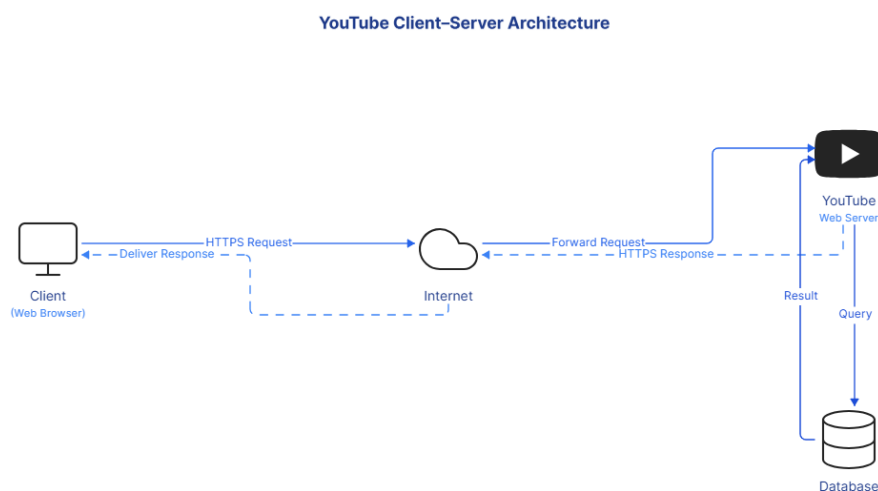


Figure 3: TCP Three-Way Handshake

Once connected, TCP guarantees reliable delivery of data using several mechanisms. Every byte of data sent is given a sequence number, which allows the receiver to reassemble segments in the correct order even if they arrive out of sequence. The receiver sends back acknowledgements confirming which data has been received successfully, and if the sender does not receive an acknowledgement within the expected time, it assumes the segment was lost and retransmits it.

TCP also performs flow control, which prevents a fast sender from overwhelming a slower receiver by adjusting how much data can be sent before requiring an acknowledgement. In addition, congestion control monitors the overall state of the network and reduces the transmission rate when congestion is detected, which helps avoid overloading routers and links along the path. Together, these mechanisms make sure that when a video page from YouTube is requested, every piece of data reaches the browser correctly and in the right order.

5. Client–Server Architecture of YouTube

YouTube is built on a Client–Server architecture rather than a Peer-to-Peer model. In this setup, the user's browser acts as the client, and YouTube's infrastructure of web servers, application servers and databases acts as the server side. The client is only responsible for sending requests and displaying whatever the server returns, while all the heavy work of storing videos, processing requests and managing user data is centralised on the server side.

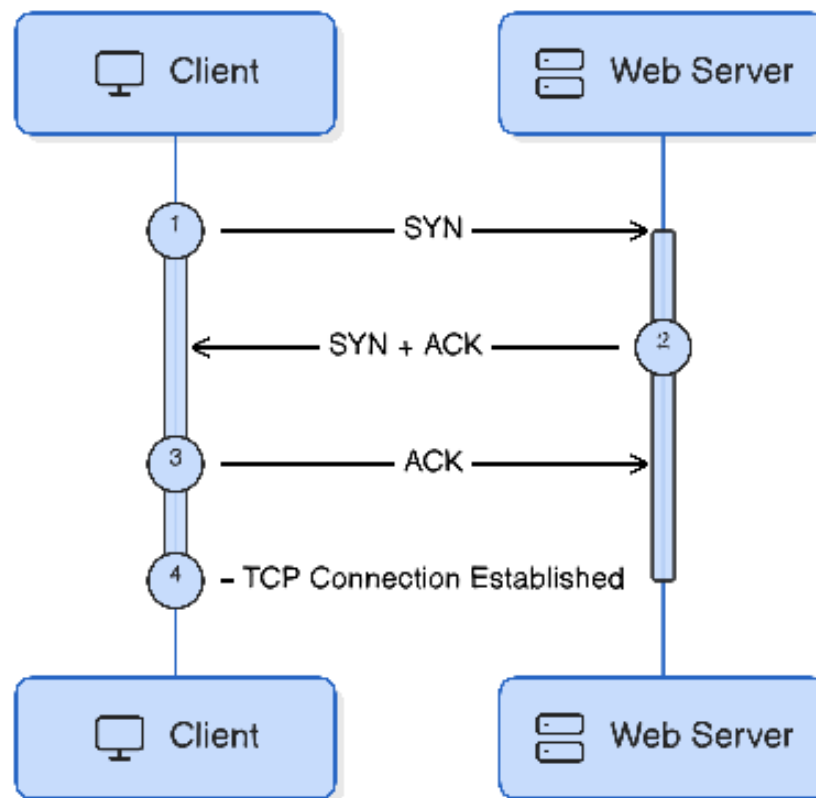


Figure 4: YouTube Client–Server Architecture

As shown in Figure 4, when the browser sends an HTTPS request, it travels through the internet to YouTube's web server. The web server forwards the request further, queries the database for the required video or page information, and returns the result back through the same path as an HTTPS response, which the browser then displays.

This architecture suits YouTube because millions of users need to access the same videos and data simultaneously. Keeping everything centralised on powerful servers makes it far easier to manage storage, apply security controls, update content, and scale the platform to serve a huge number of clients at once, something that a Peer-to-Peer model would struggle to coordinate at this scale.

6. Comparison Between HTTP and FTP

Although both HTTP and FTP are application-layer protocols used to move data over a network, they are designed for different purposes and behave quite differently, as summarised below.

Aspect	HTTP	FTP
Purpose	Used to access and display web pages and web resources.	Used mainly to transfer files between a client and a remote system.
Port Number	Uses port 80 for HTTP and port 443 for HTTPS.	Uses port 21 for control and port 20 for data transfer.
Security	HTTPS version encrypts data using TLS; plain HTTP is unencrypted.	Standard FTP sends data, including credentials, in plain text.
Authentication	Often does not require login; may use cookies, tokens or session-based authentication.	Usually requires a username and password before file access is allowed.
Data Transfer	Uses a single connection to exchange requests and responses.	Uses two separate connections, one for control commands and one for actual data.
Typical Usage	Browsing websites like YouTube, loading pages, images and videos.	Uploading or downloading files to and from a file server.

7. Role of Cookies in Improving User Experience

Cookies are small pieces of data that a server sends to the browser, which the browser then stores and sends back with every future request to that same website. Since HTTP and HTTPS are stateless by nature, meaning the server does not remember previous requests on its own, cookies act as a way to maintain continuity between the client and the server across multiple visits.

On YouTube, cookies play a big role in personalising the experience. When a user logs in, YouTube stores a session cookie so that the user stays logged in while moving between pages without having to re-enter credentials every time. Cookies also keep track of watch history, which YouTube uses to understand viewing patterns and generate personalised video recommendations on the homepage. Similarly, if a user sets a preferred language or region, this preference is stored in a cookie so that YouTube automatically loads the interface in that language on the next visit, instead of asking the user to select it again.

In this way, cookies allow YouTube to feel consistent and personalised for each user, rather than treating every request as if it were coming from a completely new visitor.

8. Complete Communication Flow

Figure 5 brings together everything discussed so far into one continuous flow, tracing what happens from the moment a user types the YouTube address until the homepage finally appears on screen.

How the Internet Delivers a Web Page

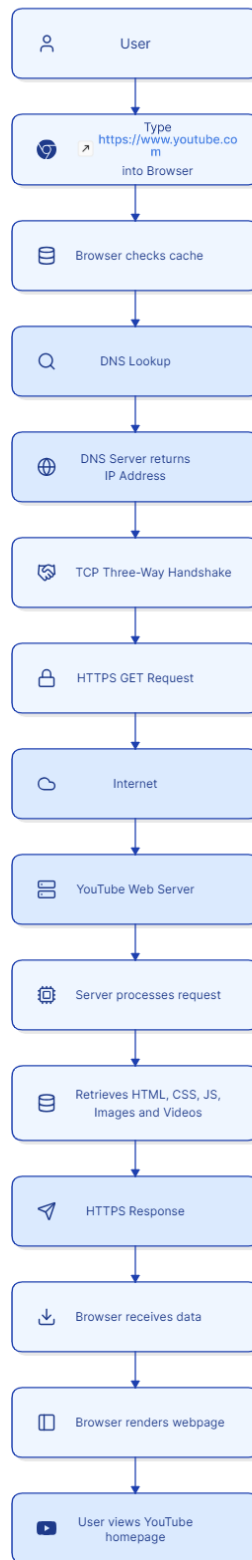


Figure 5: Complete Communication Flow for Loading YouTube

The process begins when the user types `https://www.youtube.com` into the browser. The browser first checks its cache to see if it already has the page or the resolved IP address stored from before. If not, it performs a DNS lookup, and the DNS server responds with the correct IP address of YouTube's web server. With the IP address known, the browser and server perform a TCP three-way handshake to establish a reliable connection.

Once the connection is ready, the browser sends an HTTPS GET request, which travels across the internet to reach YouTube's web server. The server processes this request and retrieves the necessary HTML, CSS, JavaScript, images and video data required to build the page. This data is packaged into an HTTPS response and sent back across the internet to the browser.

On receiving the response, the browser renders the webpage by combining the HTML structure, applying CSS styling and running any JavaScript, resulting in the fully loaded YouTube homepage being displayed to the user. What looks like a single instant action on screen is therefore the result of several coordinated steps happening across multiple layers of the network within a fraction of a second.

9. Conclusion

Application-layer protocols form the backbone of everything we do on the internet in daily life. DNS quietly translates the names we type into the addresses computers actually need, HTTPS carries our requests securely to servers like YouTube and brings back the pages, images and videos we want to see, and cookies allow websites to remember who we are between visits instead of treating us as strangers every time. None of this would work smoothly without the lower layers of the TCP/IP stack, where TCP ensures that data arrives complete and in order, and the internet and network access layers handle addressing and physical transmission. Studying how these protocols work together, as shown through the example of loading YouTube, makes it clear that even a simple action like opening a webpage depends on a well-organised chain of network communication operating behind the scenes.